

Getting Started Guide

This guide will help you install RNGNeeds, build your first weighted list, and understand how the inspector works — all in about ten minutes.

**BEST FOR**

First-time users of the plugin

TIME

About 10 minutes

CORE IDEA

Build and pick from `ProbabilityList<T>`

USE CASES

Loot tables, dice, spawns, dialogue, AI decisions

What is RNGNeeds?

RNGNeeds is a Unity plugin that lets you design **weighted probability lists** directly in the Inspector. Instead of scattering random logic across your codebase, you define the odds visually and let designers and programmers share the same source of truth.

It is built for systems where outcomes should feel random yet remain **intentional and controllable** — loot drops, enemy spawns, dialogue choices, card draws, AI decisions, and much more.

Installation

- 1 Open **Window → Package Manager** in Unity.
- 2 Select **My Assets** from the left sidebar and find **RNGNeeds - Probability Distribution**.
- 3 Click **Import**. When the import dialog appears, leave all files checked and confirm.
- 4 Verify the install by opening **Edit → Preferences → RNGNeeds** on Windows, or **Unity → Settings → RNGNeeds** on macOS.

Good to know: RNGNeeds installs as a **Unity package**, not as regular Assets. You will find it under `Packages/` in your Project window. You will also see **Starphase Core** listed as a dependency — this is expected and required.

Quick Start — A Biased Die

The best way to learn RNGNeeds is to build something small. Let's create a six-sided die where the value **6** shows up a little more often than the rest.

Step 1 — Write the Script

Create a new C# script called `Die`. All you need is a `ProbabilityList<int>` field and a method that picks from it:

```
using UnityEngine;
using RNGNeeds;

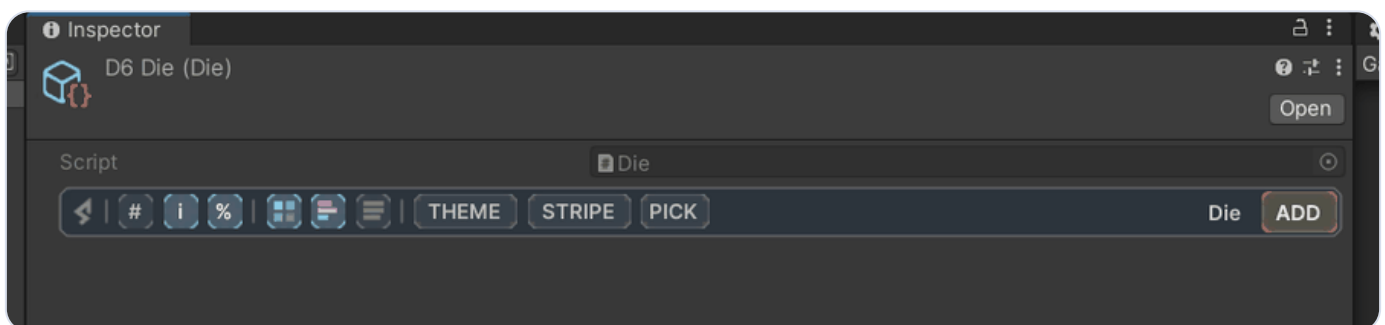
[CreateAssetMenu(fileName = "DXX", menuName = "My Die")]
public class Die : ScriptableObject
{
    public ProbabilityList<int> die;

    public int Roll()
    {
        return die.PickValue();
    }
}
```

That is the entire script. `ProbabilityList<T>` is the core type you will use throughout RNGNeeds. It holds a set of items, each with a probability, and `PickValue()` returns one result based on those weights.

Step 2 — Create the Asset

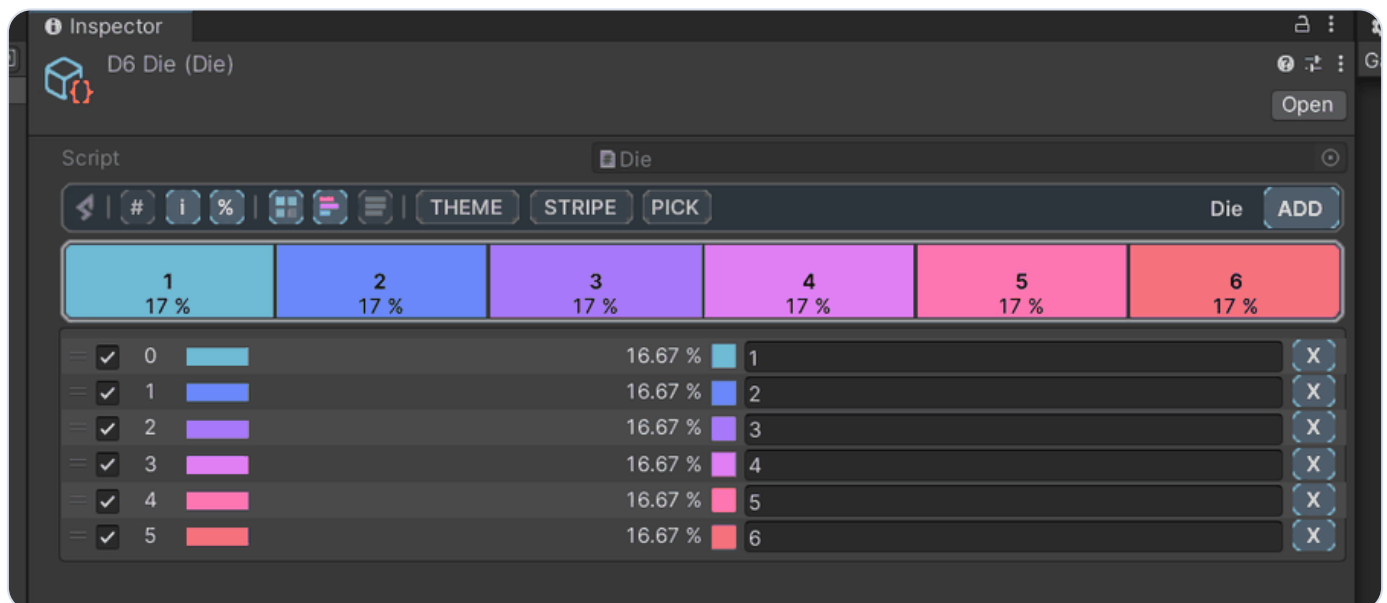
Right-click in your Project window and choose **Create → My Die**. Select the new asset and you will see an empty probability list in the Inspector, ready for items.



A freshly created Die asset with an empty probability list in the Inspector.

Step 3 — Add the Six Sides

Click the **Add** button six times to create six items, then set their values to **1** through **6**. At this point the die is perfectly fair — every item starts with an equal share of the probability.



Six sides added. Each item starts with equal probability until you adjust the distribution.

Step 4 — Tune the Bias

Click on the probability value for item **6** and type **0.20** (or **20%**), then press Enter. The remaining sides rebalance automatically. You can also drag the stripe blocks or their edges for a faster visual edit.

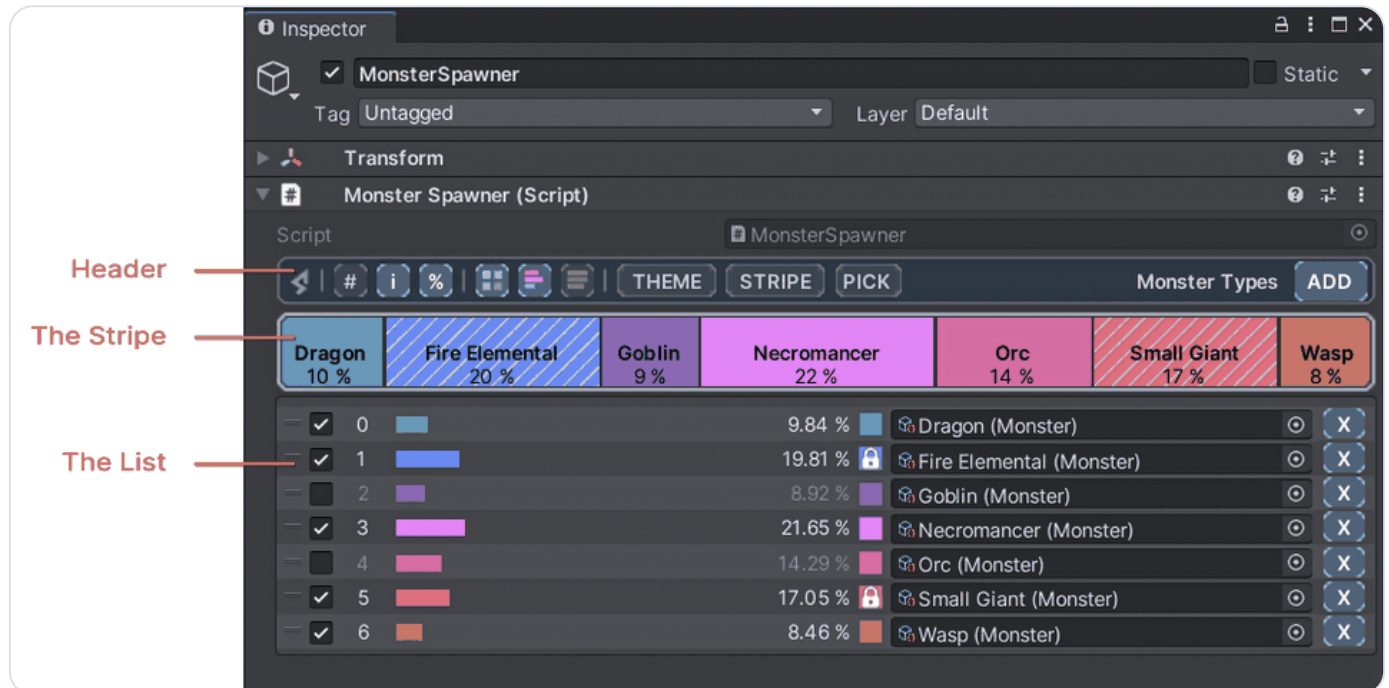


After entering the probability directly for value 6. Direct input is the most precise way to tune a bias.

That's it. Call `die.Roll()` from any script and you will get a weighted random result. Value 6 will now appear roughly 20% of the time instead of the fair ~16.7%.

Understanding the Interface

With your first list working, the next step is getting comfortable with the custom Inspector drawer. This is where you will spend most of your time designing probabilities.



The drawer has three main areas: the Header Bar at the top, the Probability Stripe in the middle, and the Item List below.

Header Bar

The top row with the property name, Add button, quick display toggles, and section controls.

Probability Stripe

A visual summary of the distribution. Drag blocks or their edges to rebalance items quickly.

Item List

The detailed editing area: values, exact probabilities, preview bars, colors, and per-item controls.

Rule of thumb: use the Stripe for fast visual balancing and the List for precise edits and item management.

Key Preferences

RNGNeeds includes editor preferences that let you tailor the drawer to your workflow. You do not need to change anything right away, but knowing these options exist will save you time later.

Drawer Options Level

Switch between **Basic** and **Advanced** controls. Basic is a fine starting point; Advanced reveals extra features like seeding, depletable list controls, and the selection method dropdown.

Drawer Option Buttons

Choose **Full**, **Compact**, or **Informative** display for the header buttons. Informative packs key settings directly into each button, so you can see the current color palette, stripe height, and pick configuration at a glance.

Inspector Refresh Mode

If the editor feels sluggish while repainting, switch to **Optimized** to repaint only during mouse drags.

Invert Scroll Direction

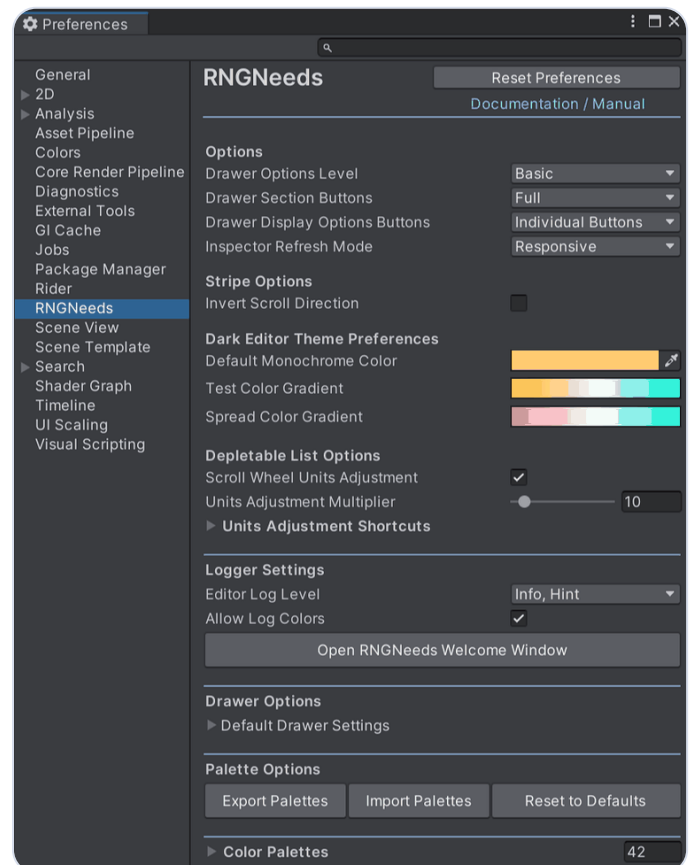
Reverses stripe scroll direction. Useful on macOS where the default "natural" scroll can feel inverted.

Default Drawer Settings

Define how newly created drawers look, so every new list starts the way you prefer.

Color Palettes

Manage shared color themes across your project for a consistent visual feel.



The RNGNeeds preferences window, available from Edit → Preferences → RNGNeeds.

Recommendation: try **Informative** option buttons right away. While the buttons show more information, seeing key settings like the pick count, repeat prevention method, and color palette at a glance is a habit worth building from the start.

What to Explore Next

You have installed the plugin, built a weighted list, and learned how the drawer works. Here are some directions to explore next.

Recommended reading

Docs

- [User Interface](#) — a deeper look at the drawer
- [Preferences](#) — every setting explained
- [Terminology](#) — the glossary of RNGNeeds concepts
- [Samples Overview](#) — ready-to-run example scenes
- [API Reference](#) — every method and type

Useful links

Full documentation
docs.rngneeds.com

Support
lab@starphase.sk

Community
[Join us on Discord](#)

Good next experiments

Loot table

Build a drop table with Common, Rare, and Legendary tiers. Tweak the weights and see how the feel changes.

Encounter list

Create a monster spawner where tougher enemies appear less often, then shift the balance and compare the results.

Sample scenes

Import the package samples and inspect how the example setups are wired in a real Unity project.

